Adversarial Game-Theoretic Security Patrol Planning with Stackelberg Equilibria

Vishal S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
vishal.s.2024.cse@rajalakshmi.edu.in

Vishaali R
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
vishaali.r.2024.cse@rajalakshmi.edu.in

***Abstract*—Fixed security patrols are predictable, and purely random patrols ignore the unequal value of targets. This paper presents a patrol planner that models a campus patrol as a Stackelberg security game. The campus is a weighted graph; a depth-first search with time and checkpoint limits enumerates all feasible routes, and a multiple-linear-programs solver computes the Strong Stackelberg Equilibrium, that is, the probability of running each route that is best for the defender against an attacker who observes it. The policy is evaluated against a perfectly rational attacker and a bounded-rational (quantal-response) attacker by Monte Carlo simulation with synchronized seeds, 95% Wilson confidence intervals and a normalized-entropy measure of unpredictability. On a six-target campus the equilibrium policy intercepts 72.4% of attacks against a rational attacker with a defender payoff of +4.48, versus 30.8% (−3.85) for uniform random patrols and 0% (−10.00) for a fixed schedule. An exact analysis shows that against softmax attackers the payoff falls toward 2.30 rather than the equilibrium value 4.43, because the equilibrium assumes a favorable tie-break. We also report four implementation defects found through end-to-end testing and the regression tests that guard against them. The system includes a web dashboard, a REST API and 64 automated tests.**

***Index Terms*—Stackelberg security games, patrol scheduling, linear programming, quantal response, bounded rationality, Monte Carlo simulation**

I. Introduction

Physical security patrols protect assets such as campuses, airports, ports and data centers with a limited number of guards. At any moment most locations are unobserved, so a patrol that follows a fixed schedule can be learned by an intruder, who then attacks where and when no one is present. Purely random patrolling removes this predictability but treats all locations alike, spending as much effort on a parking lot as on a server room.

Stackelberg security games (SSGs) address this tension [1]. The defender (leader) first commits to a randomized patrol; the attacker (follower) observes the committed distribution and responds with the most profitable attack; and the defender chooses the distribution that is best given that response (Fig. 1). The idea underlies deployed systems such as ARMOR, which randomizes checkpoints and canine patrols at Los Angeles International Airport [2], and PROTECT, which schedules Coast Guard patrols in ports [3].

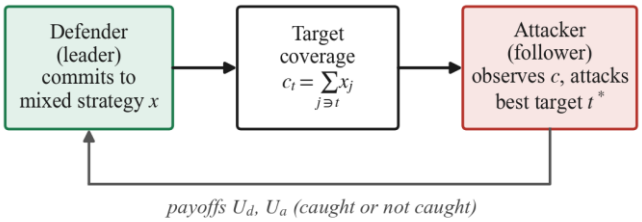


Fig. 1. Stackelberg security game: the leader commits to a mixed strategy, the follower observes it and best-responds.

This paper describes a compact, reproducible planner that applies the SSG formulation to a campus patrol and evaluates it end to end. Its contributions are: (i) a graph-based formulation in which the feasible patrol routes are generated by a constrained depth-first search; (ii) an implementation of the multiple-LPs method with a strict best-response margin that keeps the equilibrium stable under numerical rounding; (iii) a comparison with fixed and uniform-random baselines under rational and bounded-rational attackers using Monte Carlo simulation, Wilson confidence intervals and an entropy measure; (iv) an exact analysis of why the equilibrium advantage shrinks against softmax attackers; and (v) a web dashboard, REST API and 64 automated tests, including regression tests for four defects found during end-to-end testing.

The remainder of the paper is organized as follows. Section II reviews related work, Section III formulates the problem, Section IV presents the method, Section V describes the implementation, Section VI reports the experiments, and Section VII concludes.

II. Related Work

A. Security Games in Practice

Tambe [1] consolidates the theory and deployments of security games, in which a defender with limited resources commits to a randomized allocation that attackers observe and best-respond to. ARMOR [2] was deployed at Los Angeles International Airport to randomize vehicle checkpoints and canine patrol routes so that adversaries cannot predict security activity. PROTECT [3] applied the approach to Coast Guard patrols in ports with a more realistic model of the adversary. These systems motivate the objective of replacing a

predictable schedule by a randomized one that is computed rather than guessed.

B. Computing Optimal Commitments

Conitzer and Sandholm [4] showed that the leader's optimal mixed strategy to commit to can be computed by solving one linear program per pure strategy of the follower, each program requiring that strategy to be the follower's best response, and keeping the best solution. Paruchuri et al. [5] introduced DOBSS, a single mixed-integer program for Bayesian Stackelberg games in which the attacker has several possible types. Kiekintveld et al. [6] gave a compact representation for security games with a very large number of patrol assignments that avoids enumerating joint schedules. We use the construction of [4] directly, because the number of targets is small, and we discuss [5] and [6] as extensions.

C. Bounded Rationality

McKelvey and Palfrey [7] proposed quantal response equilibrium, in which players choose better actions with higher probability rather than always the best one, with a parameter controlling how sharply choices follow payoffs. Yang et al. [8] studied human adversaries in security games and found that a defender strategy computed with a quantal-response model outperformed one that assumed perfect rationality when facing human subjects. We therefore evaluate the equilibrium policy against both attacker types.

III. Problem Formulation

A. Campus Graph and Feasible Routes

The campus is an undirected graph $G = (V, E)$. Each node $t \in V$ is a target with a value v_t, and each edge $e \in E$ has a travel time w_e in minutes. A patrol *route* is a simple path $r = (u_1, \ldots, u_m)$ that visits at most K checkpoints ($m \le K$) with total travel time at most the shift duration D. Let $R = \{r_1, \ldots, r_N\}$ be the set of feasible routes. A single patrol unit follows one route per shift.

B. Payoffs

Each target has an attacker reward $R_a(t)$ if an attack succeeds, an attacker penalty $P_a(t)$ if the attacker is caught, a defender reward $R_d(t)$ if the attack is caught, and a defender penalty $P_d(t)$ if it succeeds. The defender chooses a mixed strategy $x = (x_1, \ldots, x_N)$ with $\sum x_j = 1$ and $x_j \ge 0$. The coverage of target t (the probability that the patrol passes through it, where $j \ni t$ means that route j visits t) and the expected utilities are

$$c_t = \sum_{j \ni t} x_j \tag{1}$$

$$U_a(t) = c_t\, P_a(t) + (1 - c_t)\, R_a(t) \tag{2}$$

$$U_d(t) = c_t\, R_d(t) + (1 - c_t)\, P_d(t) \tag{3}$$

The default campus used throughout is given in Table I; its corridors are Main Gate–Corridor (2 min), Corridor–Laboratory (3), Corridor–Admin Office (4), Laboratory–Server Room (2) and Admin Office–Parking Area (3). All values are synthetic and chosen only for demonstration.

TABLE I
Targets and Payoffs of the Default Campus

Target	v	R_a	P_a	R_d	P_d
Main Gate	9	9	−2	9	−9
Corridor	2	2	0	2	−2
Laboratory	8	8	−2	8	−8
Admin Office	6	6	−2	6	−6
Server Room	10	10	−2	10	−10
Parking Area	4	4	−1	4	−4

C. Strong Stackelberg Equilibrium

The attacker observes x and attacks the target $t^*(x)$ that maximizes U_a; if several targets tie, the Strong Stackelberg Equilibrium (SSE) assumes the one that is best for the defender, a tie-break under which an optimal commitment exists [9]. The defender solves

$$x^* = \arg\max_x U_d(t^*(x)), \quad t^*(x) \in \arg\max_t U_a(t) \tag{4}$$

IV. Proposed Method

A. Route Generation

Feasible routes are enumerated by depth-first search with backtracking [10] from every node (Algorithm 1). A path is extended to an unvisited neighbor only if the elapsed time stays within D and the path has fewer than K nodes. The graph is stored with NetworkX [11]. The number of simple paths can grow exponentially with graph size, so enumeration is capped at 1000 routes; on the default campus with $D = 30$ min and $K = 5$ there are $N = 36$ routes.

Algorithm 1. Feasible route generation

```
S ← ∅
for each node s ∈ V do DFS((s), 0)
procedure DFS(path, τ)
    add path to S
    if |path| < K then
        for each neighbor v of last(path), v ∉ path do
            if τ + w(last(path), v) ≤ D then
                DFS(path + (v), τ + w(last(path), v))
return S
```

B. Multiple-LP Solver

For each target k the solver finds the best mixed strategy under which k is the attacker's best response, then keeps the best of the $|V|$ solutions [4]. With coverage given by (1) the program for target k is

$$\max_x \; c_k(R_d(k) - P_d(k)) + P_d(k) \tag{5}$$

$$\text{s.t.} \;\; U_a(k) \ge U_a(t) + \varepsilon, \;\; \forall t \ne k \tag{6}$$

$$\sum_j x_j = 1, \quad x_j \ge 0 \tag{7}$$

The programs are solved with the HiGHS solver [12] through SciPy [13]. An exact SSE leaves the attacker indifferent between k and other targets, so any rounding of x can flip the best response. The margin $\varepsilon = 10^{-6}$ in (6) makes the intended target strictly preferred; if no target admits a strict best response, the solver falls back to $\varepsilon = 0$. Solver noise is cleaned only by clipping values below 10^{-12} and renormalizing; coarse rounding is deliberately avoided (Section V-C).

C. Attacker Models

A *rational* attacker attacks the target maximizing U_a, with ties resolved in the defender's favor. A *bounded-rational* attacker follows quantal response [7]: it attacks target t with probability

$$q(t) = \exp(\lambda U_a(t)) / \sum_s \exp(\lambda U_a(s)) \quad (8)$$

where $\lambda \geq 0$ controls rationality: $\lambda \to 0$ gives uniform random attacks and $\lambda \to \infty$ approaches the rational attacker.

D. Baseline Policies

The *fixed* policy always runs one selected route (by default the first, which visits only the Admin Office). The *uniform* policy gives every route probability $1/N$. The *game-theoretic* policy uses the SSE probabilities x^*.

E. Simulation and Statistical Evaluation

Because the attacker observes the committed distribution, the coverage and hence the attacker's response are computed once per run. Each round samples a route from the policy, takes the attacker's target (sampled from (8) for a bounded-rational attacker), and counts an interception if the target lies on the route; the defender earns R_d or P_d accordingly. A benchmark repeats this over independent trials whose seeds are derived from a base seed, so all policies face identical random streams. Detection probability $\hat{p}$ from n rounds is reported with a 95% Wilson interval [14],

$$[\, \hat{p} + z^2/2n \pm z\sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)} \,] / (1 + z^2/n) \quad (9)$$

which remains well behaved when $\hat{p}$ is near 0, as it is for a fixed schedule. Unpredictability of a policy is the normalized Shannon entropy [15] of its route probabilities,

$$\tilde{H}(x) = -\sum_j x_j \ln x_j / \ln N \quad (10)$$

which is 0 for a fixed schedule and 1 for the uniform policy.

V. System Implementation

A. Architecture

The system (Fig. 2) has a browser dashboard and a FastAPI backend written in Python. The backend contains the graph model, route generator, SSE solver, attacker models, simulator and evaluator, and keeps the current game state in memory. It exposes REST endpoints for each step: route generation, equilibrium solution, attacker analysis, policy sampling, simulation and benchmarking. The same process serves the dashboard, so the whole system starts with a single command.

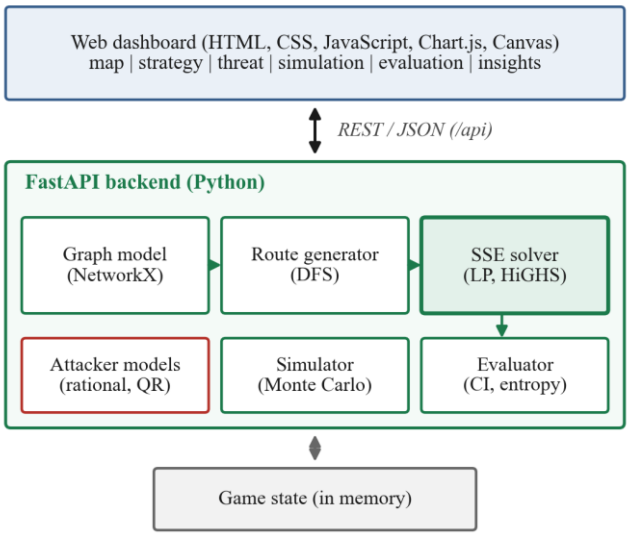


Fig. 2. System architecture of the planner.

B. Dashboard

The dashboard draws the campus graph on a canvas, with a ring and percentage at each node showing its coverage under the current equilibrium (Fig. 3), the attacker's best response circled, and a button that samples a route from the optimal policy to illustrate the randomization. A five-step progress bar tracks route generation, equilibrium, attacker profiling, simulation and benchmark. Tables and charts present the equilibrium, the attacker's expected utilities, coverage by policy, simulation logs and benchmark results, and the recommendation text is computed from the benchmark numbers rather than written in advance.

C. Verification and Defects Found

The code is covered by 64 automated pytest cases for the graph, route generation, solver, attacker models, baselines, simulation, evaluation, dashboard endpoints and the full HTTP flow. The solver was also checked against a random search over 80 000 mixed strategies at three shift lengths; it matched or exceeded the best value found in every case. End-to-end testing exposed four defects, which are fixed and covered by regression tests:

1) *Missing identifiers.* The route-generation response omitted node identifiers, so the dashboard sent route names to the solver; the coverage matrix was all zeros and the "equilibrium" was meaningless. The response now carries identifiers, and the solver also accepts names and rejects unknown checkpoints.
2) *Rounding.* Rounding probabilities to five decimals changed the attacker's best response, so the simulated payoff was 3.63 instead of the solver's 4.43. The strict margin of Section IV-B and the removal of rounding fix this.
3) *Bounded attacker.* The bounded-rational attacker always took its most likely target, making it identical to the rational attacker. It now samples from (8).
4) *Fixed route.* The route chosen for the fixed policy was ignored by the simulation and benchmark. It is now passed through the API.

None of these defects was caught by unit tests that called the solver directly; all appeared only when the dashboard flow was exercised, which motivated the API-level regression tests.

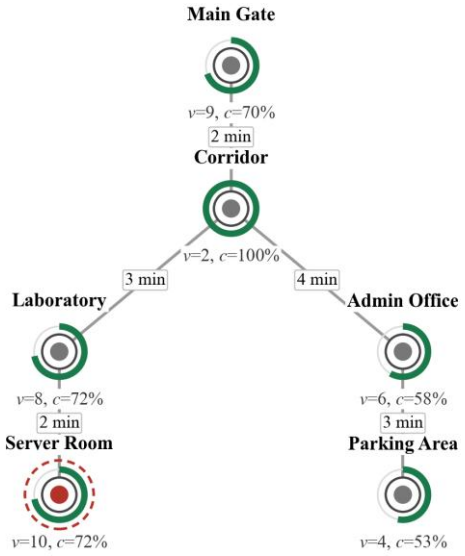


Fig. 3. Default campus under the equilibrium policy. Rings show coverage c_t; the dashed ring marks the attacker's best response (Server Room).

VI. Experiments and Results

A. Setup

Experiments use the campus of Table I with D = 30 min and K = 5, giving N = 36 routes. The benchmark runs 10 independent trials of 1000 rounds for each policy and attacker, with base seed 42 and trial i using seed 42 + 100i; the bounded attacker uses λ = 1. The six policy–attacker benchmarks (60 000 simulated rounds) complete in about 1.7 s on a laptop-class machine.

B. Equilibrium on the Default Campus

The solver returns an equilibrium with defender utility +4.43 and attacker utility +1.34, in which the attacker's best response is the Server Room. Only 4 of the 36 routes receive probability: S36 (Server Room–Laboratory–Corridor–Main Gate) 41.8%, S35 (Server Room–Laboratory–Corridor–Admin Office–Parking Area) 30.4%, S28 (Parking Area–Admin Office–Corridor–Main Gate) 22.8% and S15 (Main Gate–Corridor–Admin Office) 5.1%. The resulting coverage (Fig. 3) is highest for the Corridor (100%), which every route passes through, and about 70% for the valuable targets. The equilibrium equalizes the attacker's utility: Main Gate, Admin Office, Server Room and Parking Area all give 1.34, while the Laboratory gives 0.78 and the Corridor 0.00. No target is more attractive than the Server Room, which is exactly the condition for it to be a best response.

C. Policy Comparison

Table II and Fig. 4 compare the policies. Against a rational attacker the game-theoretic policy intercepts 72.4% of attacks (95% CI 71.5–73.3%), more than twice the 30.8% of the uniform policy, and improves the defender payoff from −3.85 to +4.48. The fixed schedule is never successful: a rational attacker simply avoids the route it knows will be patrolled. The simulated payoff +4.48 agrees with the analytic value +4.43 to within the Monte Carlo standard error of about 0.09 for 10 000 rounds.

TABLE II
Benchmark Results (10 Trials of 1000 Rounds)

Policy	Detection % [95% CI]	U_d	U_a	$\tilde{H}$
Rational attacker				
Fixed	0.0 [0.0, 0.0]	−10.00	+10.00	0.00
Uniform	30.8 [29.9, 31.7]	−3.85	+6.31	1.00
Game-theor.	72.4 [71.5, 73.3]	+4.48	+1.31	0.34
Bounded-rational attacker (λ = 1)				
Fixed	0.0 [0.0, 0.0]	−9.57	+9.57	0.00
Uniform	31.2 [30.3, 32.1]	−3.61	+5.95	1.00
Game-theor.	65.5 [64.6, 66.4]	+2.36	+1.25	0.34

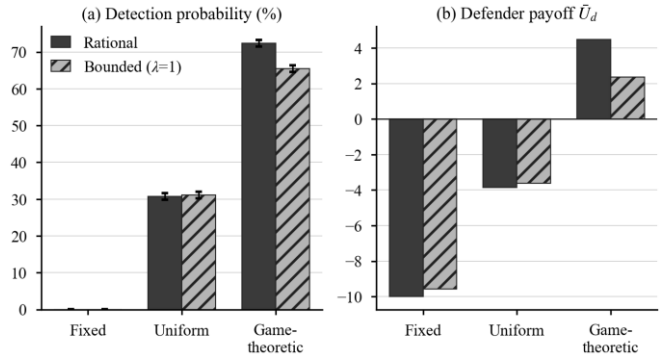


Fig. 4. Benchmark results: (a) detection probability with 95% Wilson intervals; (b) mean defender payoff.

The conclusion does not depend on the weak default fixed route. With the fixed policy set to S36, which covers four of the six targets, detection stays at 0.0% and the payoff only improves from −10.00 to −6.00: a deterministic route cannot be effective against an attacker who observes it, whichever route is chosen. The entropy column of Table II shows the price of the equilibrium's performance: it randomizes over only 4 routes (0.34), whereas the uniform policy is perfectly unpredictable (1.00) but ineffective.

D. What the Attacker Observes

Fig. 5 shows the coverage each policy gives every target, which is what the attacker observes. The uniform policy spreads coverage evenly (31% to 75%), the fixed schedule covers only the Admin Office, and the equilibrium covers all high-value targets at roughly 70% while leaving the cheap Parking Area and Admin Office less covered.

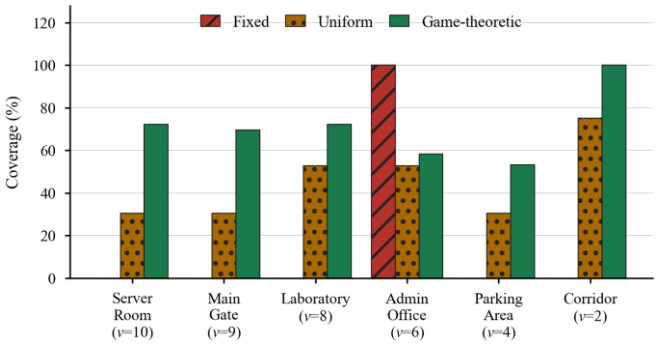


Fig. 5. Target coverage under each policy, sorted by target value v.

E. Sensitivity

Shift duration. Fig. 6(a) and Table III show that a longer shift allows longer routes and more coverage per patrol: the equilibrium payoff rises from −2.52 when only single targets can be visited to +4.43 at 15 min. Beyond 15 min the number of routes stays at 36 because the checkpoint limit K binds, so the payoff no longer changes.

TABLE III
EQUILIBRIUM VERSUS SHIFT DURATION (K = 5)

D (min)	**Routes**	**Routes used**	**U_d**	**U_a**
0†	6	4	−2.52	+5.51
5	20	4	−1.14	+4.68
8	28	5	+2.00	+2.80
10	34	5	+2.94	+2.24
15	36	4	+4.43	+1.34
30	36	4	+4.43	+1.34

† Only single-checkpoint routes (the shift duration is set to zero).

Attacker rationality. Because the SSE is computed analytically, the sensitivity to λ can be evaluated exactly rather than by simulation (Table IV, Fig. 6(b)). The equilibrium policy outperforms uniform random for every λ, but its payoff against a bounded-rational attacker (about 2.43 at $\lambda = 1$) is well below the equilibrium value 4.43, and it converges to 2.30 as λ grows, even though the attacker approaches rationality. The reason is the tie-breaking assumption of the SSE. The equilibrium makes 4 targets equally attractive, and a softmax attacker divides its attacks equally among them instead of choosing the one best for the defender. The limit is the defender's average payoff over the tied targets, 2.30, not the best of them. The benchmark value in Table II (+2.36) matches the exact 2.43 within the standard error of about 0.07.

TABLE IV
EXACT EXPECTED PAYOFF VERSUS ATTACKER RATIONALITY

λ	**Game-theor. U_d**	**Det. %**	**Uniform U_d**	**Det. %**
0.1	+2.46	70.2	−1.58	42.6
0.5	+2.45	68.1	−2.96	34.4
1	+2.43	66.3	−3.62	31.2
2	+2.39	64.5	−3.81	30.6
5	+2.32	63.4	−3.88	30.6
20	+2.30	63.3	−3.89	30.6
Rational	+4.43	72.2	−3.89	30.6

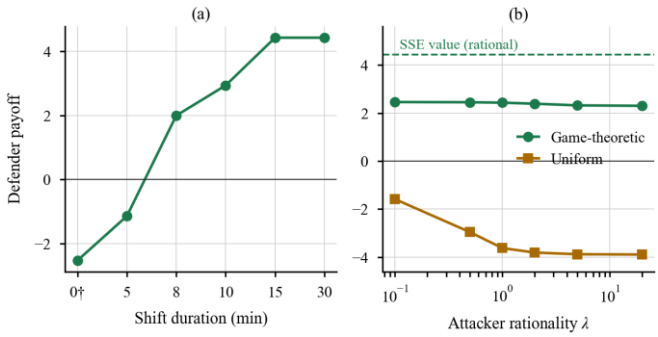


Fig. 6. Sensitivity: (a) equilibrium payoff versus shift duration († single-checkpoint routes only); (b) expected payoff versus attacker rationality λ.

F. Limitations

The study uses one small synthetic campus with a single patrol unit, so the absolute numbers are illustrative; the qualitative findings (a randomized, value-weighted patrol beats both baselines, and the equilibrium's guarantee depends on the tie-break) follow from the model rather than from the particular values. Route enumeration scales poorly with graph size, the attacker's payoffs are assumed known, and the demonstration server keeps its state in memory for a single user.

VII. CONCLUSION AND FUTURE WORK

We presented a planner that models a campus patrol as a Stackelberg security game, generates feasible routes by constrained depth-first search, computes the optimal randomized patrol by multiple linear programs, and evaluates it by simulation against rational and bounded-rational attackers. On the default campus it intercepts 72.4% of attacks against a rational attacker, against 30.8% for uniform random patrols and 0% for a fixed schedule. An exact analysis showed that the equilibrium's advantage shrinks against softmax attackers because it depends on a favorable tie-break, and end-to-end testing exposed four defects that unit tests alone had missed.

Future work includes multiple patrol units with a compact coverage-based formulation [6], several attacker types through a Bayesian Stackelberg formulation [5], an equilibrium that is robust to ties and to attacker modeling error, patrol timing in addition to location, learning the attacker's behavior from incident data, and import of real building maps.

REFERENCES

[1] M. Tambe, *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge, U.K.: Cambridge Univ. Press, 2011.

[2] J. Pita *et al.*, "Deployed ARMOR protection: The application of a game theoretic model for security at the Los Angeles International Airport," in *Proc. 7th Int. Conf. Autonomous Agents Multiagent Syst. (AAMAS), Industry Track*, 2008, pp. 125–132.

[3] E. Shieh *et al.*, "PROTECT: A deployed game theoretic system to protect the ports of the United States," in *Proc. 11th Int. Conf. Autonomous Agents Multiagent Syst. (AAMAS)*, 2012, pp. 13–20.

[4] V. Conitzer and T. Sandholm, "Computing the optimal strategy to commit to," in *Proc. 7th ACM Conf. Electron. Commerce (EC)*, 2006, pp. 82–90.

[5] P. Paruchuri, J. P. Pearce, J. Marecki, M. Tambe, F. Ordóñez, and S. Kraus, “Playing games for security: An efficient exact algorithm for solving Bayesian Stackelberg games,” in *Proc. 7th Int. Conf. Autonomous Agents Multiagent Syst. (AAMAS)*, 2008, pp. 895–902.

[6] C. Kiekintveld, M. Jain, J. Tsai, J. Pita, F. Ordóñez, and M. Tambe, “Computing optimal randomized resource allocations for massive security games,” in *Proc. 8th Int. Conf. Autonomous Agents Multiagent Syst. (AAMAS)*, 2009, pp. 689–696.

[7] R. D. McKelvey and T. R. Palfrey, “Quantal response equilibria for normal form games,” *Games Econ. Behav.*, vol. 10, no. 1, pp. 6–38, 1995.

[8] R. Yang, C. Kiekintveld, F. Ordóñez, M. Tambe, and R. John, “Improving resource allocation strategy against human adversaries in security games,” in *Proc. 22nd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2011, pp. 458–464.

[9] B. von Stengel and S. Zamir, “Leadership games with convex strategy sets,” *Games Econ. Behav.*, vol. 69, no. 2, pp. 446–457, 2010.

[10] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[11] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python Sci. Conf. (SciPy)*, 2008, pp. 11–15.

[12] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018.

[13] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.

[14] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[15] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.